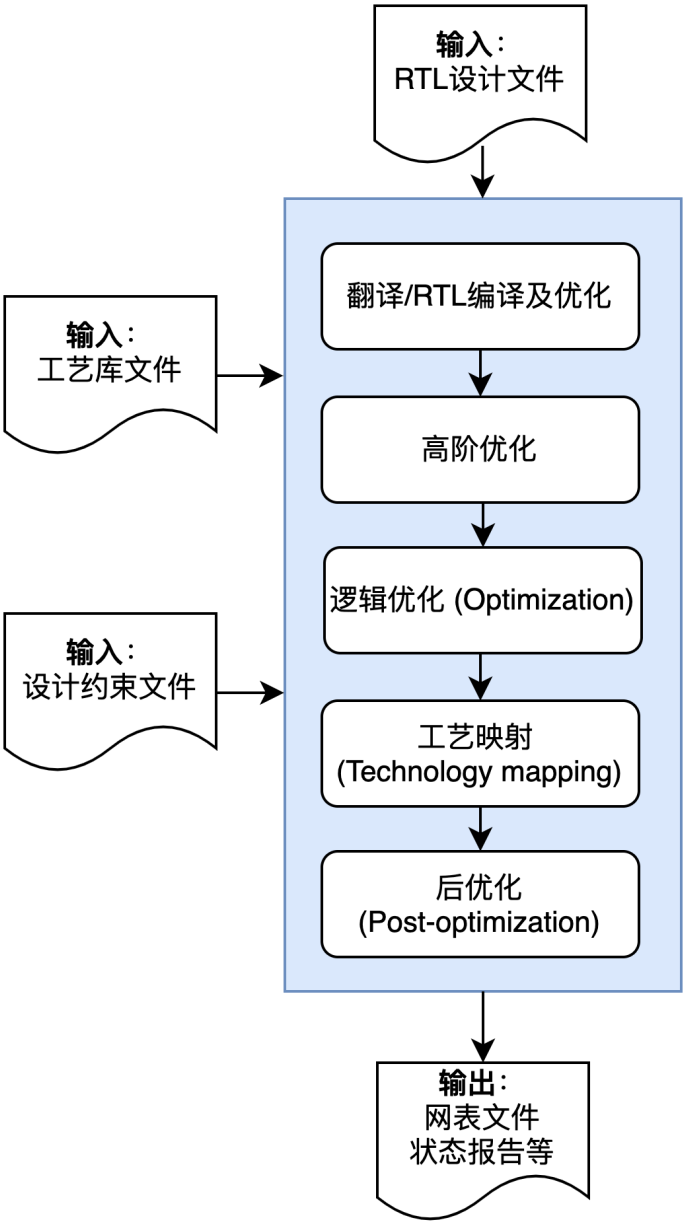
逻辑综合流程
翻译
高阶优化
常数传递优化
逻辑优化
工艺映射
参考

逻辑综合流程

逻辑综合(logic synthesis)是将用户所设计数字电路的高抽象级描述，经过布尔函数**化简**、**优化**后，转换到逻辑门级别的电路连线网表的过程。从RTL到工艺库网表的过程一般需要经历**翻译**、**RTL优化**、**逻辑优化**、**工艺映射和后优化**（post optimization）等阶段。



1. **翻译**：RTL编译及优化是将HDL转换成通用网表，同时在编译过程中进行优化；

2. **高阶RTL优化**：高阶优化是对**通用网表**的典型电路和运算符进行优化和实现；
3. **逻辑优化**：逻辑优化是对**通用网表**基于布尔代数进行化简，主要优化通用网表的节点个数、逻辑层级、开关活动性等；
4. **工艺映射**：将优化后的电路映射到制造商提供的包含具体工艺信息（面积、时延、功耗、负载等）的库上，输出工艺相关的门级网表。其中ASIC工艺映射主要是标准单元库、FPGA工艺映射是查找表；
5. **后优化**：后优化指的是工艺映射后的网表进行重新映射，运算符重新实现，buffer插入和器件尺寸选择等优化。为达到最好的PPA，通常需要引入时序约束，逻辑综合工具会根据时序要求，优化关键路径，在延时、面积和功耗中达到最佳平衡。

高阶RTL优化和逻辑优化有什么区别？

翻译

翻译是将RTL文件转换为**GTECH（generic technology）网表**，翻译的实质是一个HDL编译器，包括词法分析、语法分析和语义分析、中间表示优化、网表生成、网表优化等步骤。

> **GTECH（Generic Technology）网表** 是逻辑综合工具（如 Synopsys Design Compiler）在生成最终技术相关（technology-dependent）门级网表之前的一种中间表示形式。它是一种技术无关的门级网表，用来表示设计的逻辑结构，而不依赖于特定的工艺库（standard cell library）。
>
> GTECH使用一组通用的逻辑元件，如 `GTECH_AND2`、`GTECH_XOR2`、`GTECH_NOT` 等。每个逻辑元件都作为一个独立的模块，并通过信号连接。

1. **词法分析**：从左到右逐行扫描HDL源文件，识别分解出词法单元标记（token）关键词（例如module、assign等），标识符（例如fir_filter、clock_1p5等），常量（例如12、4'b1010等），逻辑操作/运算符（例如+、*、<<、&&等）。
2. **语法分析和语义分析**：分析token序列形成满足语法结构的中间表示，比如module，变量，类型（wire还是reg，位宽表示），always块，敏感列表，赋值语句，循环语句，常数表达式，逻辑表达式，算术表达式等。语义分析是进行一些检查，比如变量的类型是否正确，位宽是否匹配，下标是否越界，敏感列表是否包含应有的信息之类的检查。
3. **中间表示优化**：针对中间表示的优化，常见的有常数优化，运算优化，循环优化等。比如若有表达式的值在循环中不会改变，就可以把它放到循环外面，这样表达式只需要计算一次，即只生成一份逻辑电路，而不是通过循环生成多份，从而达到大幅优化的目的。
4. **网表生成**：转换中间表示形成通用网表，比如逻辑表达式转换成逻辑操作电路，运算表达式转换成算术运算单元，边沿触发的always块里可以生成寄存器，case语句的变量赋值转换成多路选择器（MUX）等。
5. **网表优化**：模块中各个always块和assign赋值分别生成网表后，这些网表形成整个模块的通用网表；同时也带来一些优化的机会，像常数，冗余提供了进一步优化的机会。

高阶优化

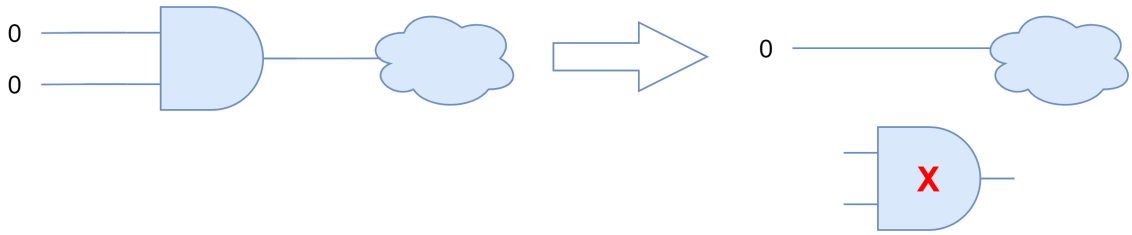
高阶优化主要是基于通用网表的优化，涉及到以下内容：

- 常数传递并优化。（yosys中`opt_expr`）
- 冗余逻辑消除。（yosys中`opt_clean`）
- 算术运算降维，比如乘法变成移位，或乘法变成加法。
- 算术运算聚类，比如加法树的提取与平衡，加乘聚类。
- 公共子表达式消除，比如输入完全相同的同类型电路模块合并为一个模块。（yosys中`opt_share`）
- 位宽优化，比如缩小总线的位宽，删除无意义的多余位宽。（yosys中`wreduce`）
- 资源共享，将对运算结果的选择转换成操作数的选择，使得运算符被共享，从而减少面积。（yosys中`share`）
- 状态机优化，比如状态最小化，可以减少用户写的状态机里的状态数目。（yosys中`fsm`）

- 状态重编码，使用最优的编码方式给状态机重编码以达到优化面积和时序的目的等等。

常数传递优化

使用常数替代电路结构，比如一个逻辑与门如果输入a和b都是常数，结果也是常数，可以直接用一个常数来替代这个与门。



逻辑优化

逻辑优化是针对现有的**逻辑表达**进行化简，在逻辑功能一致的前提下优化逻辑函数的逻辑深度、开关活动性、文字数（Literals）或者节点个数，它们分别对应电路的性能、功耗和面积，统称为PPA优化。

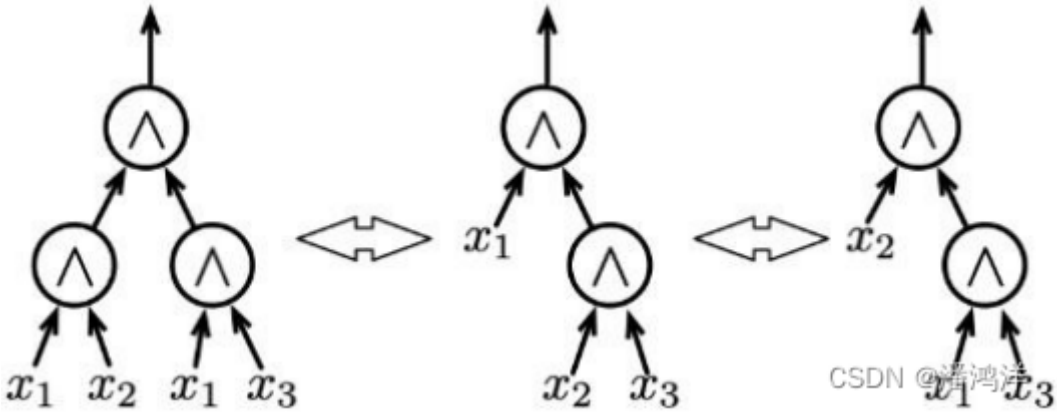
> **逻辑表达**是对数字电路中**布尔逻辑关系**的数学或符号化描述，它用于定义电路的行为及其输出与输入之间的关系。逻辑表达主要有真值表、卡诺图、析取范式、合取范式、BDD（有向无环图）和多级逻辑表达等。

逻辑优化总体分为**代数法、布尔法、RM极性变换法和精确综合法。**

- 代数法：将逻辑代数中变量看成普通代数变量，执行抽取、替代、分解、重写（Algebraic Rewriting）等优化步骤。
  - 抽取：从多个表达式中抽取相同的因子。例如$f = x_1x_2x_3x_4$和$g = x_1x_2x_5x_3x_5$的相同因子为$q = x_1x_2x_3$，则抽取后的表达式如下式所示。

$$\begin{aligned} f &= qx_4 \\ g &= qx_5 \\ q &= x_1x_2x_3 \end{aligned}$$

  - 替代（ABC中`resub`）：利用现有的表达式作为一个额外输入来替代被优化函数的输入。例如逻辑函数$f = x_1x_3 + x_1x_4 + x_2x_3 + x_2x_4 + x_5$和$g = x_1x_2$，则尝试用$g$作为$f$的一个输入。该例中$g$为$f$的因子，可以达成该目标，即$f = g(x_3x_4)x_5$。注意替代中$g$作为因子已经存在，而抽取则需要去搜寻因子。
  - 分解（ABC中`refactor`）：将一个较大规模函数分解成两个较小规模函数。函数分解实现的形式有很多种，一种最直接的方法是通过代数分解，例如将一个函数写成$f = dqr$形式，其中为d因子，q为商，r为余数。例如$f = x_1x_2x_4x_2x_3x_4x_5$，通过引入一个新变量$d = x_1x_2x_2x_3$，则$q = x_4$，$r = x_5$。分解可以以递归的形式持续进行，与替代不同的是，因子构成了一个新的变量。
  - 重写（ABC中`rewrite`）：重写基于代数公理系统在逻辑网络中执行变换，这里的代数包含一般代数和布尔代数，由于历史原因，曾经被称为代数重写。在AIG中，考虑代数等式$(x_1x_2)(x_1x_3) = x_1(x_2x_3) = x_2(x_1x_3)$，通过下图所示构造相应的AIG不难发现，可通过灵活的变换优化AIG的节点个数或者信号的逻辑深度。



- 布尔法：主要用到函数的无关项集合（Don't Care set），以此为基础将一个函数转换为另外一个函数但是不影响函数的输出结果。
- 还有RM（Reed-Muller）极性变换、精确综合等方法。

工艺映射

工艺映射是将通用网表转换成指定工艺的标准单元网表。可以分为FPGA工艺映射和ASIC工艺映射。

参考

https://blog.csdn.net/weixin_47805009/article/details/127676146